Household Energy Consumption Forecasting

A Comparative Study of ARIMA, Prophet, and XGBoost for Short-Term Load Forecasting

Project Report

Prepared by: Waleed Qamar

Github: https://github.com/waleed719/Energy-Consumption-Time-Series-Forecasting

Date: July 2026

Table of Contents

1. Introduction & Objective

Accurate short-term forecasting of household electricity consumption is valuable for grid operators, utility providers, and smart-home automation systems. This project builds an end-to-end forecasting pipeline on a real-world household power consumption dataset, comparing a classical statistical approach (ARIMA), a decomposition-based approach (Prophet), and a modern machine learning approach (XGBoost).

The project is organized into five stages, each corresponding to a notebook in the pipeline:

- Data Overview — initial profiling of the raw dataset
- Data Cleaning — parsing, missing-value handling, and hourly resampling
- Exploratory Data Analysis (EDA) — visualizing trends, seasonality, and distributions
- Modeling — feature engineering and training ARIMA, Prophet, and XGBoost
- Evaluation & Comparison — quantitative and qualitative comparison of model performance

1.1 Evaluation Metrics

Model performance is measured using two standard regression error metrics:

- Mean Absolute Error (MAE) — the average magnitude of absolute prediction errors, in the original unit (kW).
- Root Mean Squared Error (RMSE) — penalizes larger errors more heavily by squaring residuals before averaging.

2. Data Overview

The raw dataset, household_power_consumption.txt, contains minute-level household electricity readings recorded between December 2006 and November 2010, sourced from the UCI Machine Learning Repository's Individual Household Electric Power Consumption dataset.

2.1 Loading the Raw Data

The file is semicolon-delimited, with missing values encoded as the string "?". These are converted to NaN on load, and low_memory=False is used to avoid mixed dtype warnings on this 2-million-plus-row file.

```
df = pd.read_csv(
    "household_power_consumption.txt",
    sep=";",
    na_values="?",
    low_memory=False
)
```

2.2 Dataset Profile

Key characteristics identified during the initial audit:

Property	Value
Total rows	2,075,259
Total columns	9 (Date, Time, Global_active_power, Global_reactive_power, Voltage, Global_intensity, Sub_metering_1/2/3)
Missing values	25,979 rows (identical count across all power-related columns)
Duplicate rows	0
Global_active_power — mean	≈ 1.09 kW
Global_active_power — max	≈ 11.1 kW

The consistent missing-value count across columns suggests that entire rows (rather than isolated fields) are affected — likely due to short gaps in the original recording device — which supports a simple row-wise drop strategy during cleaning rather than imputation.

3. Data Cleaning & Preprocessing

The cleaning notebook transforms the raw minute-level readings into an analysis-ready hourly time series.

3.1 Datetime Construction

The Date and Time columns are concatenated and parsed into a single Datetime column. Since the source dates are formatted as DD/MM/YYYY, dayfirst=True is passed to pandas.to_datetime to avoid misparsing day/month values.

```
df["Datetime"] = pd.to_datetime(
    df["Date"] + " " + df["Time"],
    dayfirst=True
)
```

3.2 Type Coercion and Missing-Value Removal

Global_active_power is explicitly coerced to numeric (errors="coerce"), and all rows containing any NaN values are dropped. This reduces the dataset from 2,075,259 to 2,049,280 rows — a loss of roughly 1.25% of records.

3.3 Resampling to Hourly Averages

The Date and Time string columns are dropped once the Datetime index is set, and the dataset is resampled to hourly frequency using the mean of each hour's minute-level readings. This reduces noise, shrinks the dataset size substantially, and produces a series better suited to short-term load forecasting.

```
df = df.set_index("Datetime")
df.drop(columns=["Date", "Time"], inplace=True)
hourly = df.resample("h").mean()
```

The resulting hourly dataset is written to household_power_consumption_cleaned.txt and used as the input for all downstream EDA and modeling notebooks.

4. Exploratory Data Analysis

The EDA notebook visualizes the cleaned hourly series at multiple resolutions to surface trends, seasonality, and distributional characteristics before modeling.

4.1 Hourly Consumption Over Time

The full hourly series shows the raw variability of the signal across the roughly four-year recording window, including short-term spikes and longer quiet periods (e.g. vacations or unoccupied periods).

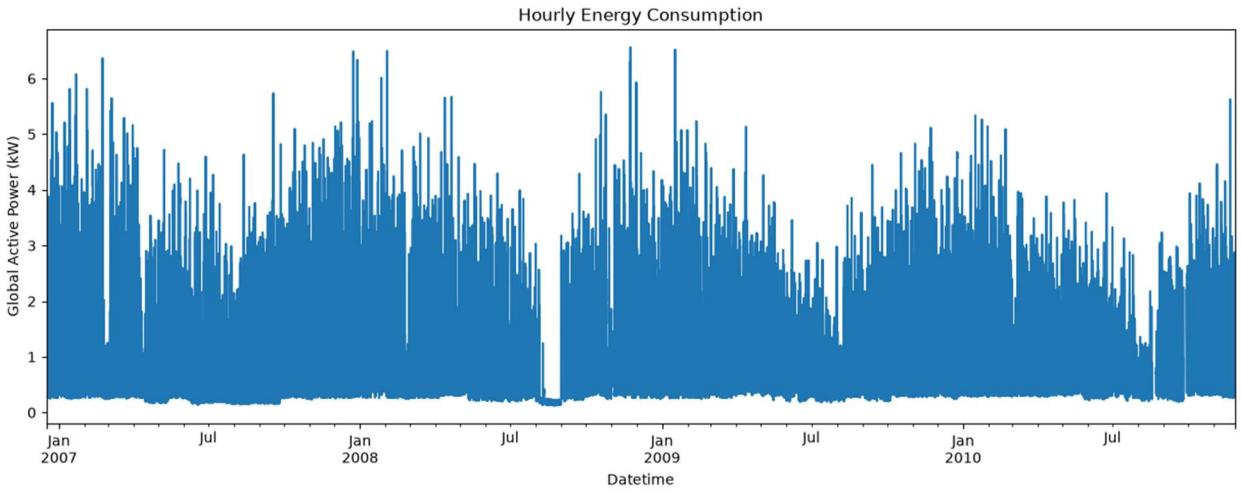

Figure 1. Hourly Global Active Power over the full observation period.

4.2 Daily Consumption Trend

Resampling to daily averages smooths out intraday fluctuations, revealing day-to-day and weekly cyclical patterns more clearly.

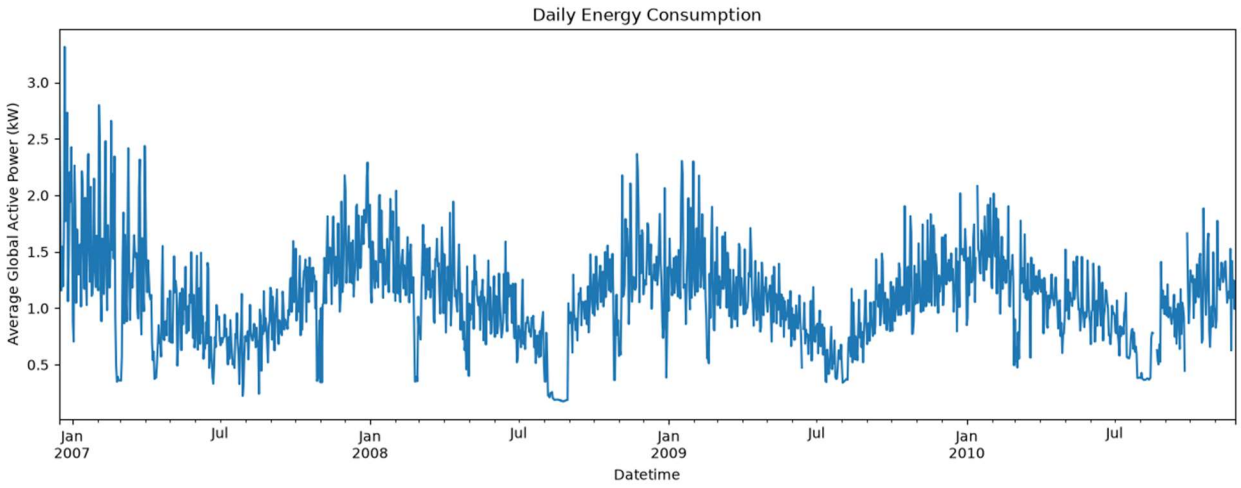

Figure 2. Daily average Global Active Power.

4.3 Monthly Consumption Trend

Aggregating to monthly averages exposes the annual seasonal cycle: consumption rises in winter months (heating and lighting demand) and dips during summer.

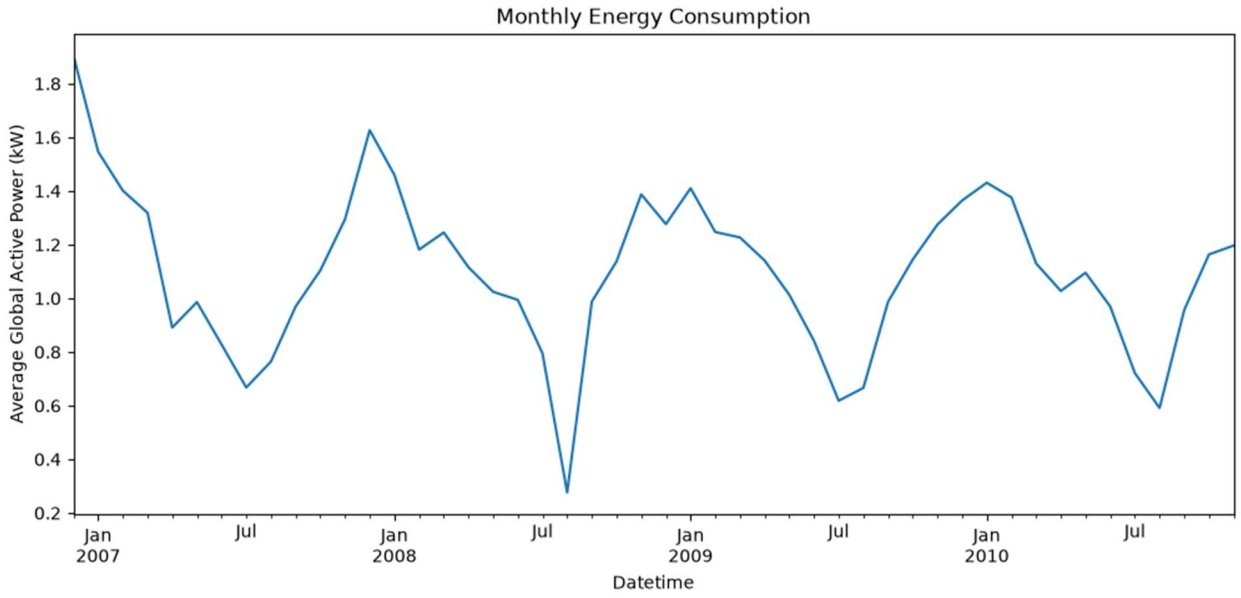

Figure 3. Monthly average Global Active Power, highlighting winter peaks.

4.4 Distribution of Global Active Power

A histogram with a kernel density overlay shows the value distribution is right-skewed, with most hourly readings clustered at lower power levels and a long tail toward higher-consumption hours.

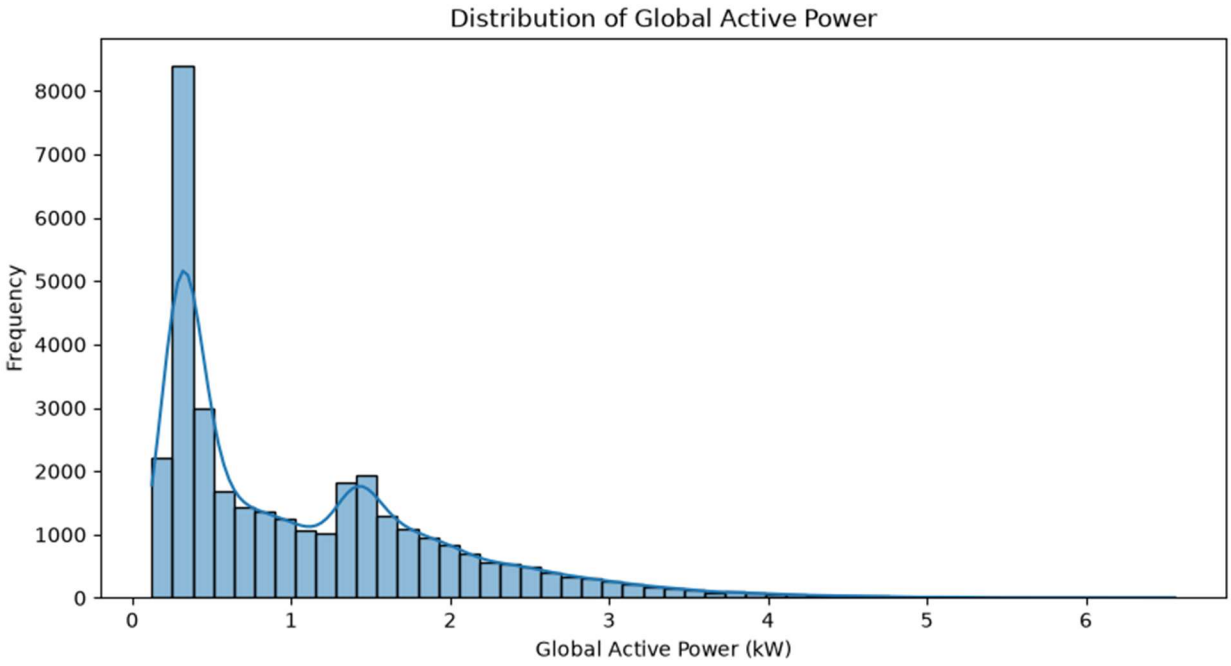

Figure 4. Distribution of hourly Global Active Power readings.

4.5 Outlier Detection via Box Plot

The box plot confirms the right-skew and highlights a substantial number of high-consumption hours beyond the upper whisker, consistent with periods of heavy simultaneous appliance use.

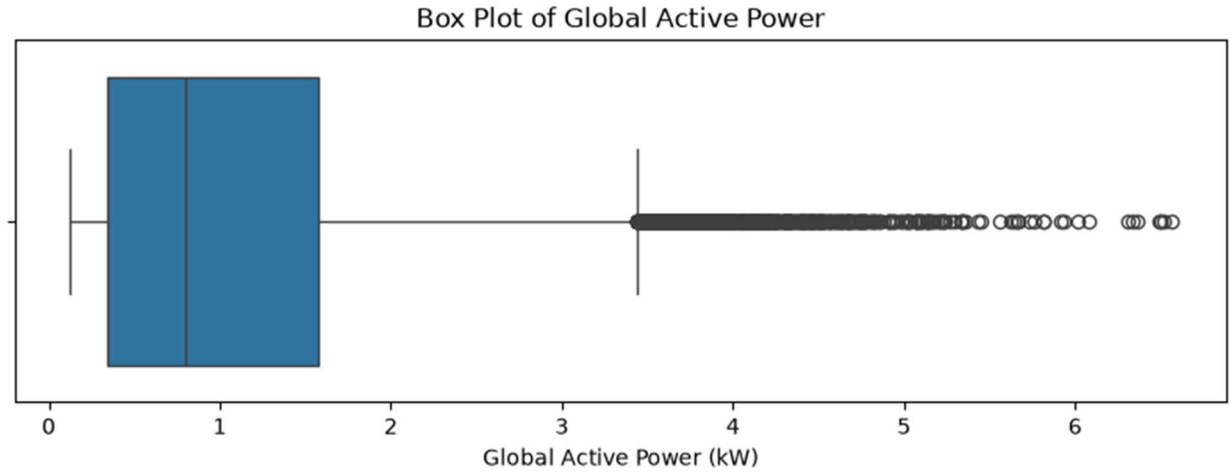

Figure 5. Box plot of Global Active Power, showing IQR and outliers.

5. Feature Engineering & Modeling

The modeling notebook engineers predictive features and trains three forecasting models on the cleaned hourly dataset.

5.1 Feature Engineering

Three categories of features are constructed:

- Temporal features: Hour, Day, Month, Weekday, and a binary Weekend indicator, extracted directly from the datetime index to capture periodic cycles.
- Lag features: Lag1 (previous hour), Lag24 (same hour, previous day), and Lag168 (same hour, previous week), capturing autocorrelation at multiple time scales.
- Rolling statistics: Rolling24, a 24-hour rolling mean capturing the recent local level of consumption.

Rows containing NaNs introduced by the lag/rolling operations (up to the first 168 hours) are dropped to produce a fully aligned feature matrix.

5.2 Train / Test Split

A sequential (non-shuffled) split is used to respect the temporal ordering of the data and avoid leaking future information into training: all but the final 500 hours form the training set, and the last 500 hours are held out for testing.

Split	Rows
Training set	32,887
Test set	500

5.3 Model 1 — ARIMA

An ARIMA(5, 1, 2) model is fit directly on the Global_active_power series (train_series), using only the target's own past values, differencing, and moving-average error terms — no external features are used.

5.4 Model 2 — Prophet

The data is reshaped into Prophet's required ds (datetime) / y (target) format. Prophet is fit on the training series and used to generate predictions for the test period's timestamps, automatically decomposing trend and seasonal components.

5.5 Model 3 — XGBoost

An XGBRegressor is trained using the full engineered feature set (Hour, Day, Month, Weekday, Weekend, Lag1, Lag24, Lag168, Rolling24). Unlike ARIMA and Prophet, XGBoost has no innate concept of time order — its ability to model temporal structure depends entirely on the engineered lag and calendar features.

5.6 Exported Artifacts

Predictions from all three models (alongside the actual test values) are exported to model_predictions.csv, and XGBoost's feature importances are exported to xgboost_feature_importances.csv, decoupling model training from evaluation.

6. Evaluation & Model Comparison

6.1 Quantitative Results

Each model's forecasts are compared against the actual held-out values using MAE and RMSE:

Model	MAE	RMSE
ARIMA	0.676	0.824
Prophet	0.549	0.701

Model	MAE	RMSE
XGBoost	0.361	0.529

XGBoost achieves the lowest error on both metrics, followed by Prophet, with ARIMA performing the weakest over this 500-hour horizon.

6.2 Forecast vs. Actual Visualization

Overlaying all three forecasts against the actual test-period consumption shows how closely each model tracks the diurnal peaks and valleys of household demand.

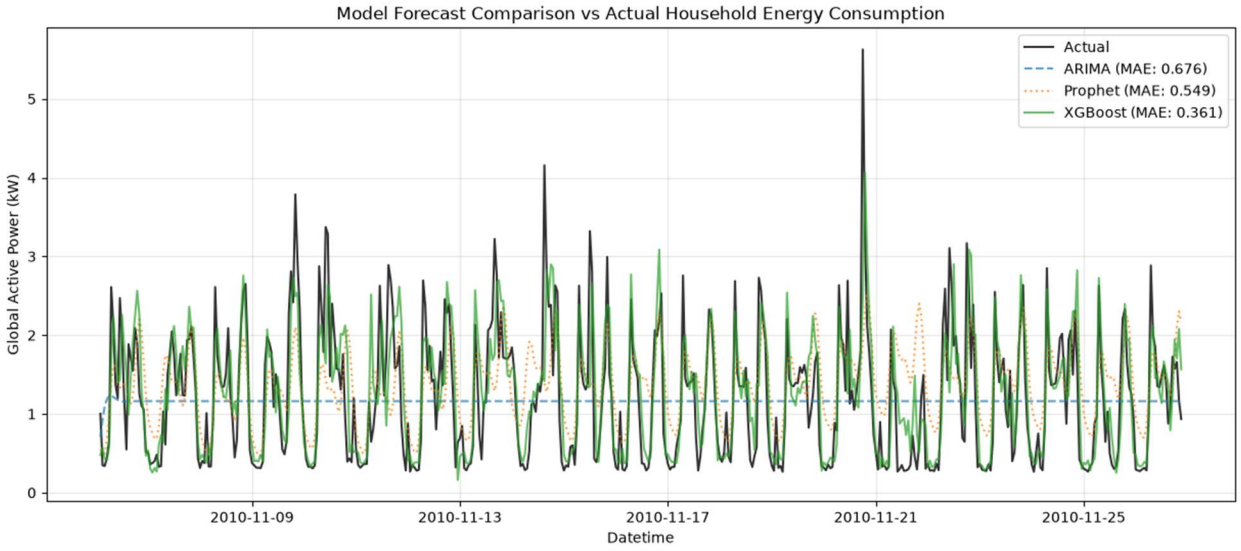

Figure 6. Actual vs. forecasted Global Active Power for ARIMA, Prophet, and XGBoost.

6.3 XGBoost Feature Importance

Inspecting XGBoost's feature importances validates that the model relies most heavily on recent and same-hour historical consumption, aligning with physical intuition about household energy use.

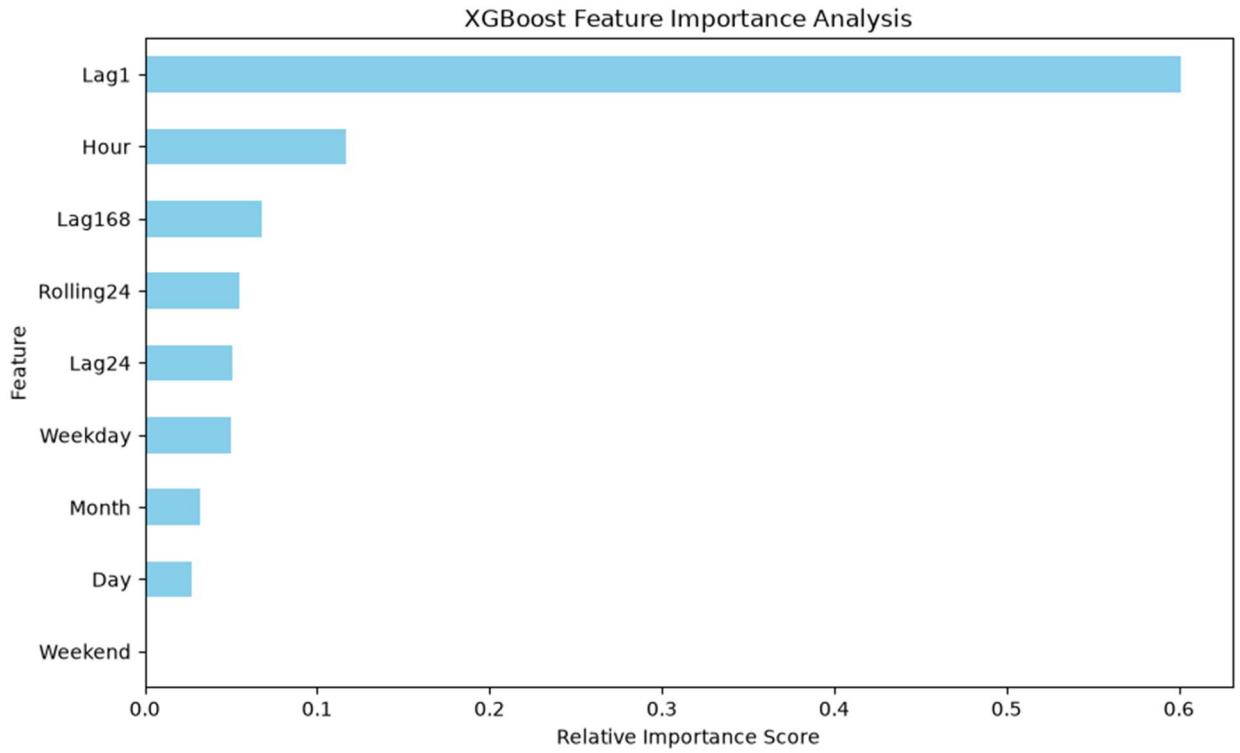

Figure 7. Relative feature importance scores from the trained XGBoost model.

6.4 Model Comparison and Trade-offs

Criteria	ARIMA	Prophet	XGBoost
Trend	Yes	Yes	Yes (learned via features)
Seasonality	Manual configuration	Automatic detection	Via temporal & lag features
Feature support	No external features	Limited covariates	Excellent support
Speed	Medium	Medium	Fast (highly scalable)
Accuracy	Moderate	Good	Often best (given enough features)

Trade-offs Discussion

- ARIMA performs moderately well for very short-term prediction but degrades over longer horizons, depending heavily on manual parameter tuning (p, d, q) and struggling to represent complex exogenous signals or multiple overlapping seasonalities (e.g., hourly and daily combined).
- Prophet requires very little tuning to capture multiple nested seasonalities (daily, weekly, yearly) and structural trends, making it robust and accessible for business applications.
- XGBoost is highly flexible and delivers the strongest forecasting precision in this study, but it requires careful feature engineering — generating lags, rolling windows, and calendar indicators — since it has no native representation of time.

7. Business Insights & Recommendations

- Strong temporal patterns: Household energy usage follows pronounced hourly patterns (evening peak demand) and weekly patterns, with distinct weekday vs. weekend demand profiles.
- Autoregressive predictive power: The single most predictive feature for energy usage is the immediately prior hour's consumption (Lag1), followed closely by consumption at the same hour on the previous day (Lag24).
- Grid load balancing: Utility operators can deploy XGBoost-based forecasting to anticipate demand spikes up to 24 hours in advance, enabling more efficient allocation of power generation resources and reduced operational costs.
- Smart appliance scheduling: Home automation systems can use short-term forecasts to schedule heavy energy consumers (e.g., heating systems, EV charging) during predicted off-peak hours, reducing electricity bills and easing grid stress.

8. Conclusion

This project demonstrates a complete forecasting workflow — from raw data ingestion through cleaning, exploratory analysis, feature engineering, and multi-model evaluation — on real household energy consumption data. Among the three approaches tested, XGBoost delivered the

most accurate short-term forecasts when supplied with well-engineered lag, rolling, and calendar features, while Prophet offered a strong, low-maintenance alternative for capturing seasonality with minimal tuning. ARIMA, though a solid statistical baseline, was the least competitive of the three on this dataset and horizon.

Future work could explore hybrid approaches (e.g., using Prophet's decomposed trend/seasonality as features for XGBoost), incorporating exogenous weather data, and extending the evaluation to multiple forecast horizons beyond the 500-hour test window used here.